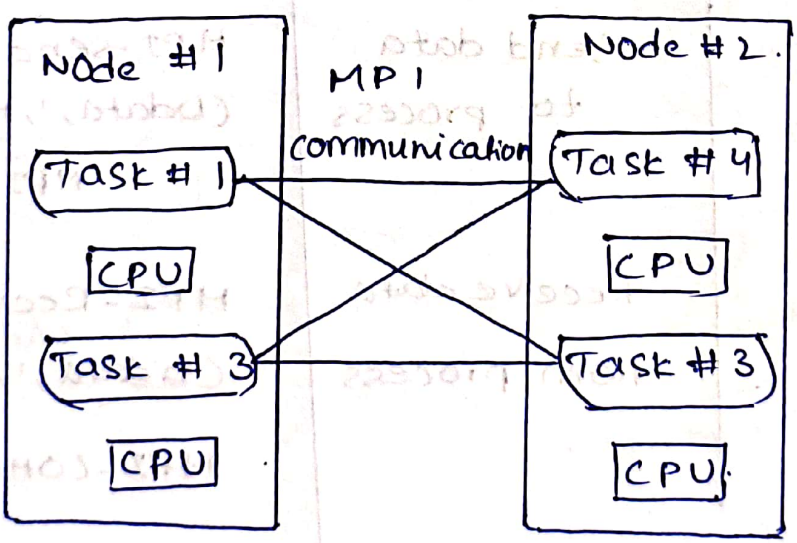
Assignment - 2

M.V.Raghupathi Sai
2023BCS0096.

Distributed Deep Learning System using MPI.

1. Introduction:-

* **Definition:-** Distributed deep learning is the process of training deep NN's across multiple machines or processors to reduce training time and handle large datasets.
* **Why MPI:-** Message Passing Interface (MPI) is a standardized and portable message-passing system that enables processes on multiple nodes to communicate efficiently.
* **Advantages:**
  - scalability
  - faster training
  - Efficient resource utilization.



Nodes communicating with each other via MPI.

Scanned with OKEN Scanner

2. Basics of MPI:-

* **Definition:-** MPI provides functions to exchange data between processes in a distributed system.

* **Key features:-**
  - point to point communication
  - collective communication
  - synchronization.

* **Common MPI functions:-**

function	purpose	Example usage
MPI_init	intialize MPI environment.	MPI_Init(&argc, &argv);
MPI_Comm_size	No. of processes	MPI_Comm_size (MPI_COMM_WORLD, &size);
MPI_comm_rank	Rank of process	MPI_comm_rank (MPI_COMM_WORLD, &rank);
MPI_Send	Send data to process	MPI_send (&data, 1, MPI_INIT, 1, 0, MPI_COMM_WORLD);
MPI_Recv.	Receive data from process	MPI_Recv (&data, 1, MPI_INIT, 0, 0, MPI_COMM_WORLD, &status);

Scanned with OKEN Scanner

3. Distributed Deep Learning approaches:-

3.1 Data parallelism:-

* Each node gets a subset of the dataset.
* Each node trains a full model locally.
* Gradients are synchronized across nodes.

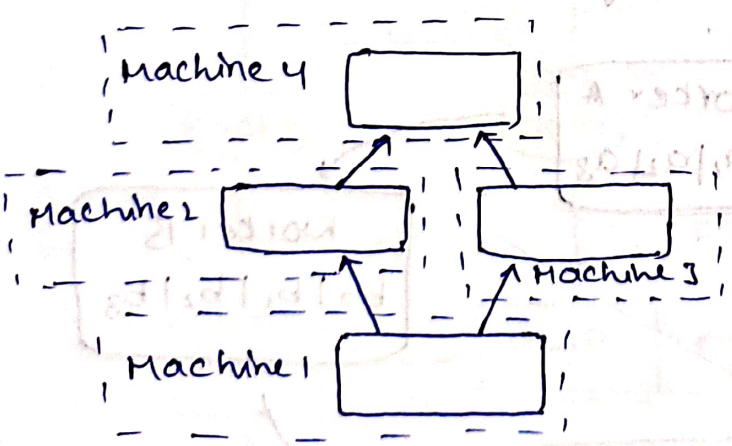
3.2 Model parallelism:-

* The model is split across nodes.
* Each node computes a part of the network.
* Useful for very large models that don't fit into a single GPU/CPU memory.

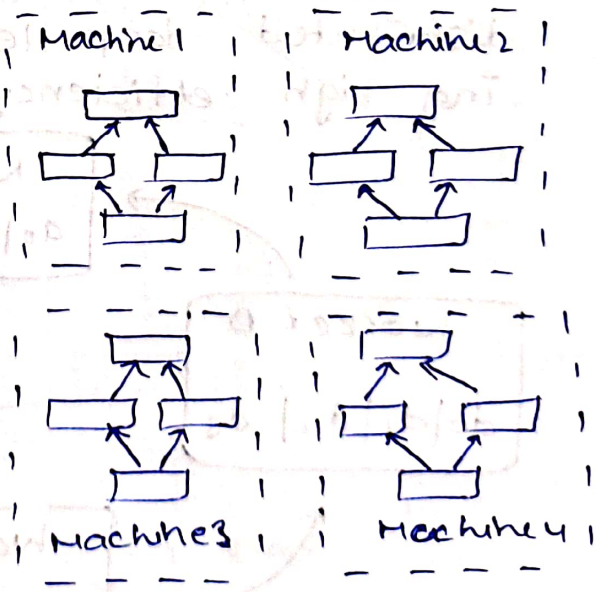
3.3 Hybrid parallelism:-

* combination of data and model parallelism
* maximizes hardware utilization.

Model Parallelism



Data parallelism.

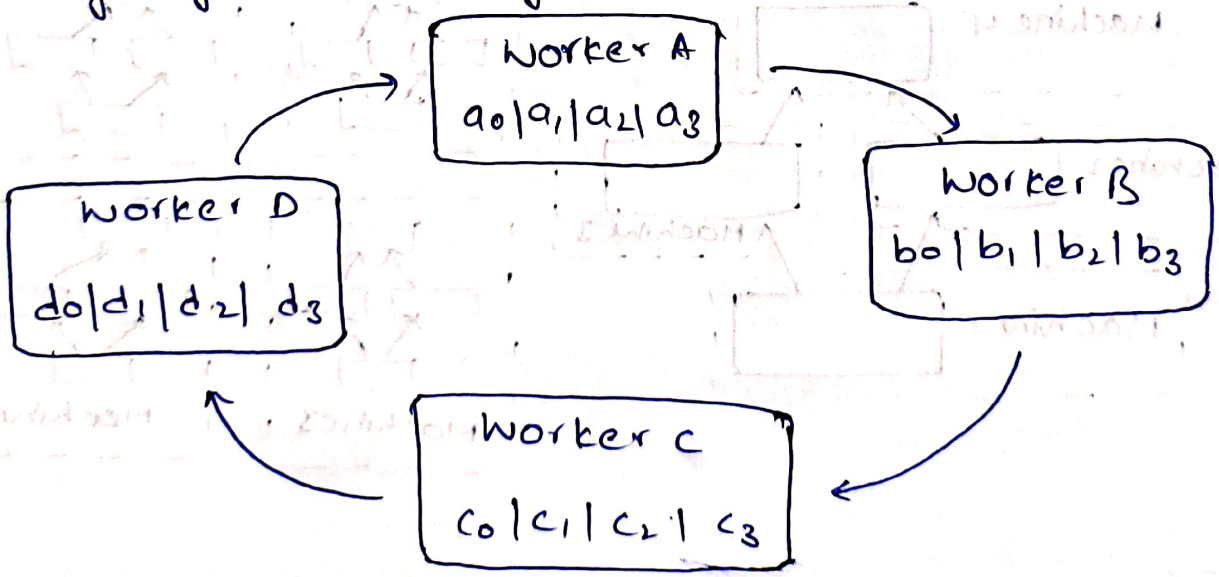


Scanned with OKEN Scanner

4. Frameworks supporting MPI:-

Framework -	MPI Support	Description.
Tensor flow	tf.distribute. MirroredStrategy, Horovod	Supports distributed training using MPI for gradient sync.
PyTorch	torch.distributed, Horovod	provides Distributed DataParallel for model /data parallelism.
Horovod	Built on MPI	Open-source library for distributed deep learning across Tensor flow, PyTorch, MXNet.
MXNet	kvstore via MPI	Supports synchronous & asynchronous distributed training.

* Horovod is widely preferred because it simplifies distributed deep learning with MPI while maintaining high efficiency.



MPI job and Horovod runtime.

Scanned with OKEN Scanner

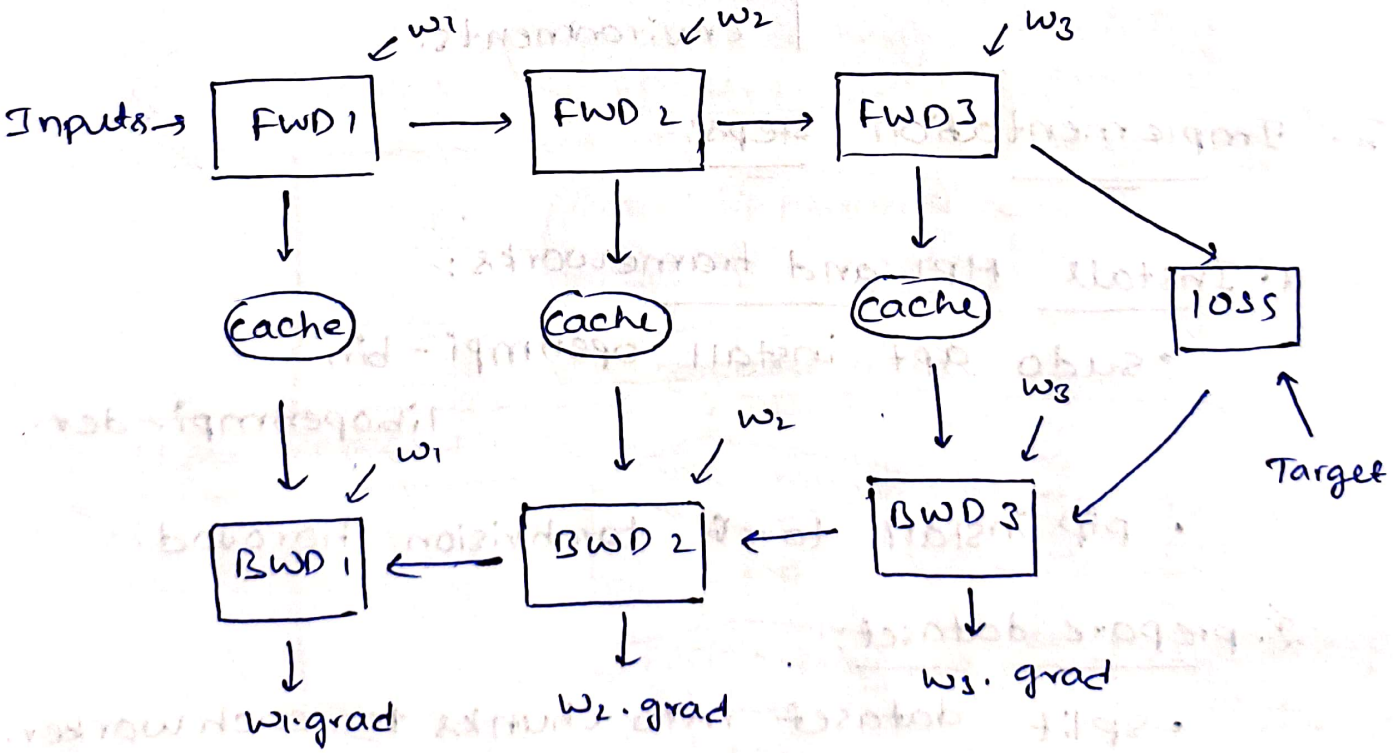
5. System Architecture Design :-

* **components:**

1. Master Node (Rank 0)
   - coordinates tasks
   - aggregate gradients.
2. worker Nodes (Rank 1, 2 ... N)
   - perform local computation
   - send gradients to master.

* **Data flow:**

- Dataset is split among worker nodes
- Each node computes forward / backward pass.
- Gradients are sent via MPI Allreduce
- updated parameters are broadcasted back.



flowchart

Outputs = FWD(input)

errors = ∇ loss wrt. layer input.

Scanned with OKEN Scanner

6. Tools and libraries:

Tool / Library	Purpose.
MPI (OpenMPI / MPICH)	communication protocol between nodes
CUDA / cuDNN	GPU acceleration for local computations.
Horovod	Gradient synchronization across nodes.
NCCL	NVIDIA collective communication library for GPU based operations.
SLURM	job scheduling in cluster environments.

7. Implementation Steps:-

1. Install MPI and frameworks:-
   - sudo apt install openmpi-bin libopenmpi-dev.
   - pip install torch torchvision horovod.
2. prepare dataset:-
   - split dataset into chunks for each worker.

Scanned with OKEN Scanner

3. Initialize MPI environment:-

* import horovod.torch as hvd

  hvd.init().

4. Distribute Model and Data:-

* Wrap model with hvd.DistributedOptimizer().

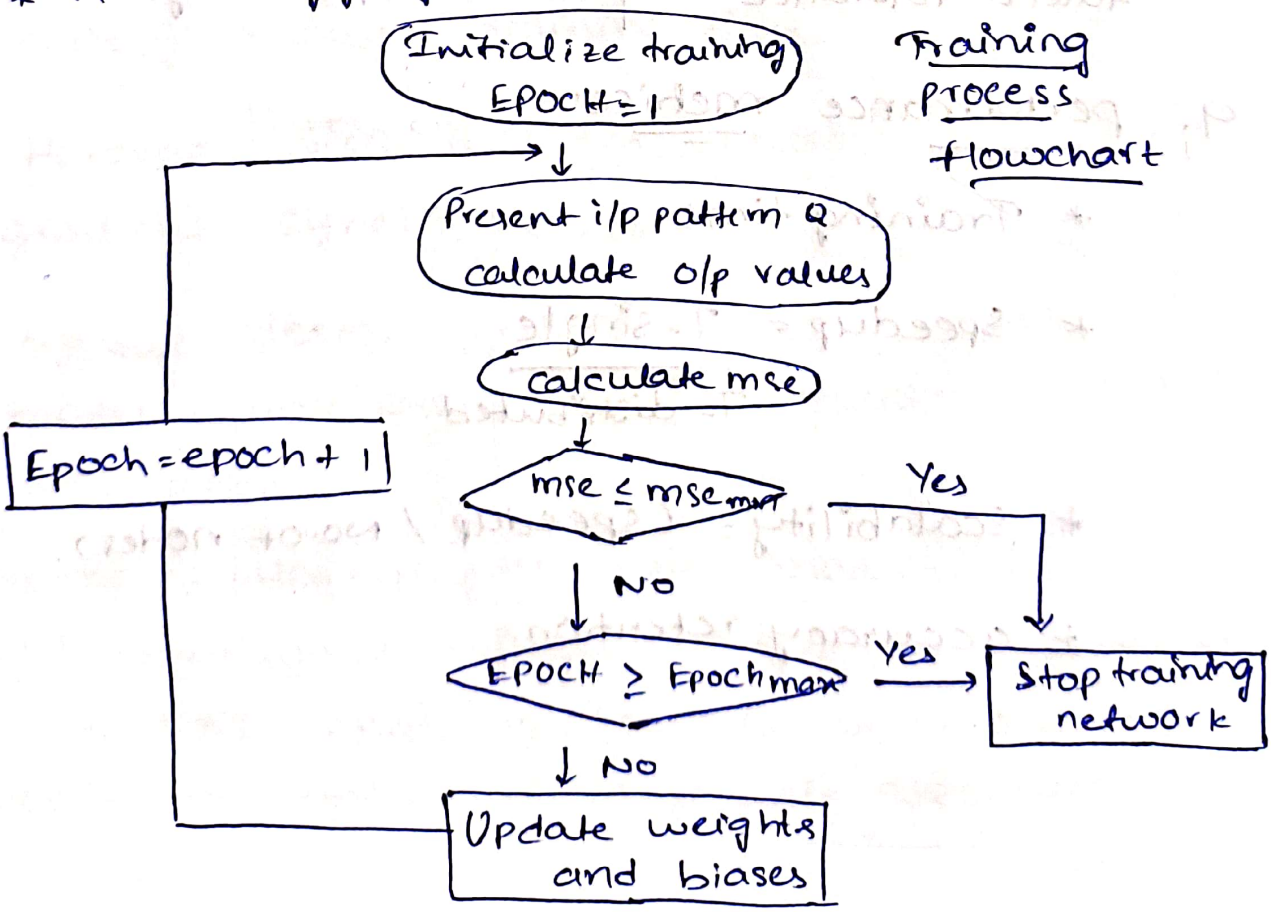
5. Training Loop:-

* forward pass
* Backward pass
* gradient synchronization via hvd.allreduce().

6. Evaluation:-

* Each node evaluates model locally.
* Master aggregate results.

Training Process Flowchart



Scanned with OKEN Scanner

8. challenges in Distribute deep learning:-

* communication overhead
* load balancing.
* synchronization issues.
* fault tolerance.

Solutions to challenges:-

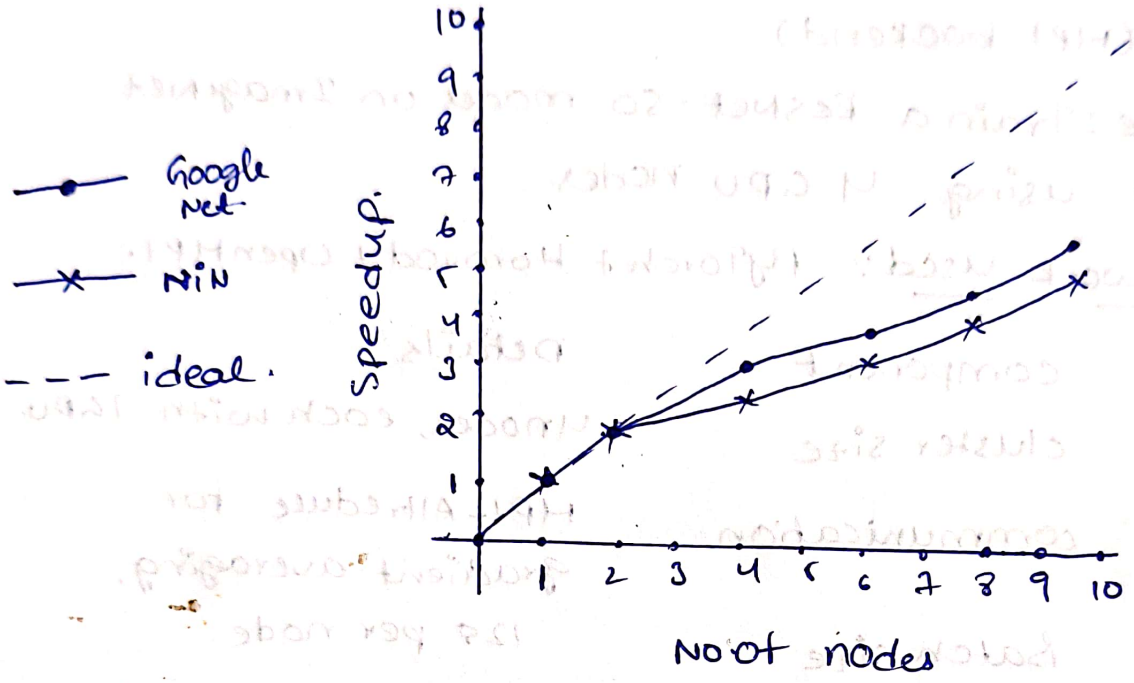
challenge	solution.
communication overhead	Use Allreduce, compression or mixed precision.
load imbalance	Dynamic data partioning
fault tolerance	checkpointing.

9. performance metrics:-

* Training time
* Speedup = $\frac{T\text{-single}}{T\text{-distributed}}$.
* Scalability = (speedup / No.of nodes).
* accuracy retention.

Scanned with OKEN Scanner

Speedup of model training versus no.of nodes:-



10. Conclusion:-

* MPI based distributed deep learning significantly reduces training time

* Horovod simplifies the process of gradient synchronization

* System design should consider data/model parallelism and hardware utilization

furthermore, integrating MPI with modern deep learning frameworks bridges the gap b/w traditional HPC and AI, enabling scalable and efficient solutions for real world large scale applications.

Scanned with OKEN Scanner

Case Study - Distributed deep learning using Horovod and MPI

case Study: Image classification using ResNet-50 with Horovod (MPI backend).

* objective: Train a ResNet-50 model on ImageNet dataset using 4 GPU nodes.

* framework used: PyTorch + Horovod + OpenMPI.

* setup:

component	Details
cluster size	4 nodes, each with 1 GPU.
communication	MPI_Allreduce for gradient averaging.
Batch size	128 per node.
Optimizer.	SGD with momentum

* process flow:-

1. Dataset split across nodes.
2. Each node trains locally for each batch.
3. Gradients averaged using Horovod's hvd.allreduce().
4. Master broadcasts updated parameters.

* Result:-

- Training time reduced by ~3.5x compared to single GPU.
- Accuracy remains similar (~76% top-1)

conclusion:- MPI based distributed training with Horovod achieves near linear scaling & efficient communication.

Scanned with OKEN Scanner